Design and Implementation of a Matrix-Based File Encryption System

- A Lattice-Inspired Matrix Encryption System for Secure File Transformation

Submitted by:
Prakhar Pandey

GitHub Repository:
https://github.com/PSquare3027/Design-and-Implementation-of-a-Matrix-Based-File-Encryption-System.git

1.Abstract

This project presents the design and implementation of a custom file encryption system developed in Python. The system represents plaintext characters as three-dimensional vectors and encrypts them using matrix transformations under modulo 257 arithmetic.

The project began with a simple fixed-matrix approach and gradually evolved through multiple versions. Later versions introduced deterministic matrix generation using a shared seed and finally block-wise matrix generation to reduce visible ciphertext repetition.

The implementation supports text file encryption and decryption while processing data in small blocks, allowing large files to be handled without loading the entire file into memory. Testing was performed on files of different sizes and structures, including empty files, partial blocks, and large text files. In every test case, the decrypted output matched the original input, confirming the correctness of the implementation.

Although the algorithm is intended primarily for educational purposes and does not claim security comparable to modern standards such as AES, the project demonstrates the practical application of matrix operations, modular arithmetic, deterministic key generation, and reversible encryption techniques.

Keywords:

Cryptography, Modular Arithmetic, Matrix Encryption,

Deterministic Key Generation, Vector Spaces,

Lattice-Inspired Encryption, File Security, Python

2. Introduction

Traditional encryption algorithms such as RSA and AES are widely used for secure communication and data protection. However, ongoing research in quantum computing has raised concerns regarding the long-term security of certain classical cryptographic systems.

This project explores a custom educational encryption system inspired by vector-space transformations, modular arithmetic, and concepts commonly associated with lattice-based cryptography. The objective was to design and implement a reversible file encryption mechanism capable of transforming plaintext into ciphertext using matrix operations under modular arithmetic.

The project focuses on understanding the mathematical foundations behind encryption systems while exploring deterministic key generation and block-wise transformations.

3. Background and Motivation

Modern encryption systems play a critical role in protecting digital information. At the same time, ongoing research in quantum computing has encouraged interest in alternative cryptographic approaches. This project was motivated by a desire to understand how mathematical concepts such as vectors, matrices, and modular arithmetic can be used to build a working encryption system.

The development of large-scale quantum computers has motivated significant research into post-quantum cryptography. Many currently deployed public-key cryptographic systems rely on mathematical problems that could become vulnerable to sufficiently powerful quantum computers.

As a result, researchers and organizations worldwide are actively investigating alternative cryptographic approaches that may remain resistant to future quantum attacks. One promising area of research is lattice-based cryptography, which relies on the computational difficulty of solving problems within high-dimensional geometric structures known as lattices.

This project does not implement a true lattice-based cryptosystem. Instead, it draws inspiration from vector-space representations, matrix transformations, and deterministic mathematical structures that are conceptually related to some ideas found within lattice-based cryptographic research.

The goal of the project is educational: to explore how mathematical transformations can be used to construct a reversible encryption system and to investigate the design challenges involved in developing secure communication mechanisms.

4. Objectives

The primary objectives of this project were:

• To design and implement a reversible file encryption system.

• To encrypt and decrypt text files without loss of information.

• To represent plaintext data as mathematical vectors.

• To utilize matrix transformations under modular arithmetic for encryption and decryption.

• To generate encryption matrices deterministically from a shared seed value.

• To reduce direct ciphertext repetition through block-wise matrix generation.

• To support large file processing through streaming techniques rather than loading entire files into memory.

• To investigate concepts inspired by vector-space and lattice-based mathematical structures.

• To document the evolution of the encryption architecture through multiple design iterations.

5. Evolution of the Encryption Architecture

The final encryption system was not designed in a single iteration. Instead, multiple versions were developed and evaluated in order to improve flexibility, scalability, and resistance to observable ciphertext patterns.

5.1 Version 1 – Static Matrix Encryption

The initial implementation used a single fixed 3×3 matrix as the encryption key.

Architecture

Plaintext
→ ASCII Conversion
→ 3-Dimensional Vector Representation
→ Matrix Transformation
→ Modular Arithmetic (mod 257)
→ Ciphertext

Advantages

- Simple implementation.
- Easy to verify mathematically.
- Fast encryption and decryption.
- Demonstrated successful reversible transformation.

Limitations

A single matrix was reused for the entire encryption session.

As a result, identical plaintext blocks always produced identical ciphertext blocks. Repeated words or patterns within a file could therefore generate observable ciphertext repetition.

This version successfully demonstrated the mathematical correctness of the encryption model but exposed structural weaknesses related to frequency analysis.

5.2 Version 2 – Seed-Based Matrix Generation

To eliminate the dependency on a hardcoded matrix, deterministic matrix generation was introduced.

Instead of storing a fixed matrix, a numerical seed was used to generate the encryption matrix dynamically.

Architecture

Seed
→ Deterministic Sequence Generator
→ Matrix Generation
→ Encryption Matrix

Advantages

- Different encryption sessions could use different matrices.
- Reduced reliance on manually selected keys.
- Improved scalability.
- Allowed deterministic reconstruction of the same matrix during decryption.

Design Rationale

Both sender and receiver possess the same seed value. Using the deterministic generation process, identical matrices can be recreated independently without transmitting the entire matrix.

This design also creates a natural path toward future RSA-based seed exchange mechanisms.

Remaining Limitation

Although different sessions now used different matrices, all blocks within the same file still used the same matrix. Consequently, repeated plaintext blocks continued to generate repeated ciphertext blocks.

5.3 Version 3 – Per-Block Matrix Generation

The final version extended the seed-based approach by generating a unique matrix for every plaintext block.

A block counter was combined with the seed value to derive a different matrix for each block processed.

Architecture

Seed + Block Counter
→ Deterministic Sequence Generator
→ Matrix Generation
→ Block-Specific Encryption Matrix

Plaintext Block
→ Matrix Transformation
→ Ciphertext Block

Advantages

- Different plaintext blocks are encrypted using different matrices.
- Reduces visible ciphertext repetition.
- Increases variability throughout the encrypted file.
- Maintains deterministic reconstruction during decryption.

Design Rationale

The sender and receiver process blocks in the same order. Therefore, the block counter remains synchronized during encryption and decryption, allowing both parties to independently generate identical matrices for each block.

This approach preserves reversibility while introducing additional diversity into the encryption process.

Verification

The final implementation was validated through repeated encryption and decryption tests. Multiple file sizes and input lengths were tested, including edge cases involving partial blocks and padding.

All test cases successfully restored the original plaintext, confirming the correctness of the design.

6. Mathematical Foundation

The encryption system is based on representing text as mathematical vectors and applying reversible matrix transformations under modular arithmetic.

6.1 Plaintext Representation

The plaintext is first converted into its ASCII representation.

For example:

Plaintext:

hot

ASCII Representation:

104, 111, 116

These three values are treated as a three-dimensional vector:

P = [104, 111, 116]

This vector becomes the fundamental unit of encryption.

6.2 Matrix-Based Transformation

A 3×3 encryption matrix is generated from a deterministic seed value.

Example:

$$K = \begin{matrix} 2 & 3 & 5 \\ 4 & 8 & 2 \\ 11 & 6 & 19 \end{matrix}$$

The plaintext vector is transformed through matrix multiplication.

Mathematically:

$C = K \times P$

where:

P = plaintext vector

K = encryption matrix

C = ciphertext vector

6.3 Modular Arithmetic

To keep values within a finite numerical range and ensure reversibility, all calculations are performed under modular arithmetic using:

Prime Modulus = 257

The ciphertext vector is therefore calculated as:

$C = (K \times P) \bmod 257$

The use of modular arithmetic prevents uncontrolled numerical growth while preserving the mathematical structure required for decryption.

6.4 Matrix Invertibility

For successful decryption, every encryption matrix must possess an inverse under modulo 257 arithmetic.

This condition is satisfied only when:

$\det(K) \neq 0 \pmod{257}$

where:

det(K) represents the determinant of the matrix.

Matrices that do not satisfy this condition are rejected during matrix generation.

6.5 Decryption Process

Decryption uses the modular inverse matrix:

K^{-1}

The original plaintext vector is recovered using:

$P = (K^{-1} \times C) \bmod 257$

Because K^{-1} is the inverse of K under modulo 257 arithmetic, the original plaintext vector is reconstructed exactly.

6.6 Deterministic Matrix Generation

Rather than storing a fixed matrix, matrices are generated deterministically from a shared seed value.

A deterministic numerical sequence is produced from the seed and used to populate matrix elements.

Because the same seed always produces the same sequence, both encryption and decryption can independently reconstruct identical matrices without transmitting the matrix itself.

This mechanism forms the basis of the project's key-generation process.

6.7 Block-Wise Matrix Generation

In the final implementation, matrix generation is performed for each plaintext block.

The matrix generation seed is derived from:

Seed + Block Counter

This produces a unique matrix for every block while maintaining deterministic reconstruction during decryption.

Consequently, different portions of the plaintext are transformed using different matrices, reducing direct ciphertext repetition and increasing variability throughout the encrypted file.

6.8 Reversibility Verification

The correctness of the mathematical model was verified through repeated encryption and decryption tests.

For a plaintext vector P:

P → Encryption → Ciphertext → Decryption → P

Successful recovery of the original plaintext confirms the correctness of the matrix transformation, modular arithmetic, and inverse matrix calculations used throughout the system.

7. Implementation Details

The encryption system was implemented in Python 3 using only standard language features and built-in modules. The implementation was designed to remain lightweight while supporting deterministic key generation, modular matrix operations, and streaming file processing.

7.1 Matrix Determinant Calculation

The determinant function is responsible for determining whether a generated matrix is invertible under modulo 257 arithmetic.

A matrix can only be used for encryption if its determinant is non-zero modulo 257.

Purpose:

• Verify matrix invertibility.

• Prevent generation of invalid encryption matrices.

Function:

deter()

```
def deter(m):
    return (
    m[0][0] * (m[1][1] * m[2][2] - m[1][2] * m[2][1])
    - m[0][1] * (m[1][0] * m[2][2] - m[1][2] * m[2][0])
    + m[0][2] * (m[1][0] * m[2][1] - m[1][1] * m[2][0])
    )
```

7.2 Modular Matrix Inversion

The inverse_matrix_mod() function computes the modular inverse of a 3×3 matrix.

The determinant is first calculated and then inverted using modular arithmetic. The adjugate matrix is subsequently constructed and multiplied by the determinant inverse to obtain the inverse matrix.

Purpose:

• Enable decryption.

• Recover the original plaintext vectors.

Function:

inverse_matrix_mod()

```python
def inverse_matrix_mod(m, mod=PRIME):
    det = deter(m) % mod
    det_inv = pow(det, -1, mod)
    c = [[
            (m[1][1] * m[2][2] - m[1][2] * m[2][1]),
            -(m[1][0] * m[2][2] - m[1][2] * m[2][0]),
            (m[1][0] * m[2][1] - m[1][1] * m[2][0])
        ],[
            -(m[0][1] * m[2][2] - m[0][2] * m[2][1]),
            (m[0][0] * m[2][2] - m[0][2] * m[2][0]),
            -(m[0][0] * m[2][1] - m[0][1] * m[2][0])
        ],[
            (m[0][1] * m[1][2] - m[0][2] * m[1][1]),
            -(m[0][0] * m[1][2] - m[0][2] * m[1][0]),
            (m[0][0] * m[1][1] - m[0][1] * m[1][0])
        ]]
    adj = [[c[j][i] for j in range(3)]for i in range(3)]
    inverse = []
    for row in adj:
        new_row = []
        for value in row:
            new_row.append((value * det_inv) % mod)
        inverse.append(new_row)
    return inverse
```

7.3 Deterministic Sequence Generator

A deterministic numerical sequence is generated from a user-provided seed.

The sequence generator ensures that identical seeds always produce identical numerical streams.

Purpose:

• Deterministic key generation.

• Reproducible matrix construction.

Function:

sequence()

```
def sequence(seed):

    x = seed

    while True:
        x = (
            x * x
            + 17 * x
            + 31
        ) % 2147483647
        yield x
```

7.4 Dynamic Matrix Generation

Matrices are generated dynamically from the deterministic sequence.

Generated values populate a 3×3 matrix, which is then tested for invertibility before being accepted.

Purpose:

• Eliminate hardcoded encryption keys.

• Allow reproducible matrix generation from a seed.

Function:

gen_matrix()

```
def gen_matrix(seed):

    seq = sequence(seed)

    while True:
        matrix = [
            [
                (next(seq) % 50) + 1
                for _ in range(3)
            ]
            for _ in range(3)
        ]
        determinant = deter(matrix) % PRIME
        if determinant:
            return tuple(tuple(row)for row in matrix)
```

7.5 Matrix-Vector Transformation

The matrix_vec_multi() function performs matrix multiplication between a plaintext vector and an encryption matrix.

All results are reduced modulo 257.

Purpose:

• Perform the primary encryption transformation.

• Perform the inverse transformation during decryption.

Function:

matrix_vec_multi()

```python
def matrix_vec_multi(matrix, vector):

    result = []

    for row in matrix:
        total = 0
        for i in range(3):
            total += row[i] * vector[i]
        result.append(total % PRIME)

    return result
```

7.6 File Encryption Process

The encryption process operates directly on text files.

Workflow:

1. Read plaintext characters from the input file.
2. Convert characters into ASCII values.
3. Group ASCII values into three-dimensional vectors.
4. Generate a block-specific matrix using the shared seed and block counter.
5. Apply matrix transformation under modulo 257 arithmetic.
6. Store the resulting ciphertext in a binary output file.

The seed value is written into the output file header to allow reconstruction during decryption.

Purpose:

• Transform plaintext into ciphertext.
• Support large files without loading the entire file into memory.

Function:

encrypt_file()

```
def encrypt_file(in_path, out_path, seed):
    block_counter = 0

    with open(in_path, "r", encoding="utf-8") as inp,\
        open(out_path, "wb") as out:
        out.write(seed.to_bytes(8, "big"))
        block = []
        while True:
            char = inp.read(1)
            if not char:
                break
            block.append(ord(char))
            if len(block) == 3:
                key = gen_matrix(seed + block_counter)
                cipher = matrix_vec_multi(key, block)
                block_counter += 1
                for value in cipher:
                    out.write(value.to_bytes(2, "big"))
                block.clear()

        if block:
            while len(block) < 3:
                block.append(0)
            key = gen_matrix(seed + block_counter)
            cipher = matrix_vec_multi(key, block)
            block_counter += 1
            for value in cipher:
                out.write(value.to_bytes(2, "big"))
```

7.7 File Decryption Process

The decryption process reconstructs the matrix sequence using the stored seed and block counter.

Workflow:

1. Read the seed from the ciphertext file.
2. Reconstruct the corresponding matrix.
3. Compute the modular inverse matrix.
4. Transform ciphertext vectors back into plaintext vectors.

5. Convert ASCII values back into characters.
6. Reconstruct the original text file.

Purpose:

• Recover original plaintext.
• Verify correctness of the encryption process.

Function:

decrypt_file()

```
def decrypt_file(in_path, out_path):
    block_counter = 0

    with open(in_path, "rb") as inp:
        seed = int.from_bytes(inp.read(8), "big")
        with open(out_path, "w", encoding="utf-8") as out:
            while True:
                block = []
                for _ in range(3):
                    raw = inp.read(2)
                    if not raw:
                        break
                    block.append(int.from_bytes(raw, "big"))
                if len(block) != 3:
                    break
                key = gen_matrix(seed + block_counter)
                inv_key = inverse_matrix_mod(key)
                plain = matrix_vec_multi(
                    inv_key,
                    block
                )
                block_counter += 1
                for value in plain:
                    if value != 0:
                        out.write(chr(value))
```

7.8 Memory-Efficient Processing

The implementation was designed as a streaming system.

Instead of loading the entire file into memory, data is processed block-by-block.

Advantages:

• Reduced memory consumption.
• Scalability for larger files.
• Consistent memory usage regardless of file size.

This design allows encryption and decryption to operate on files significantly larger than the available working memory.

7.9 Final Implementation Architecture

Input Text File
↓
ASCII Conversion
↓
Vector Formation
↓
Seed + Block Counter
↓
Matrix Generation
↓
Matrix Transformation
↓
Modular Arithmetic
↓
Ciphertext File

Ciphertext File
↓
Seed Recovery
↓
Matrix Reconstruction
↓
Modular Inverse Matrix
↓
Vector Recovery
↓
ASCII Reconstruction
↓
Original Text File

8. Testing and Results

The encryption system was subjected to multiple tests to verify correctness, reversibility, and reliability under different input conditions.

The primary objective of testing was to ensure that encrypted files could be decrypted back to their original form without any loss of information.

8.1 Round-Trip Verification

The most important validation test performed was round-trip verification.

Procedure:

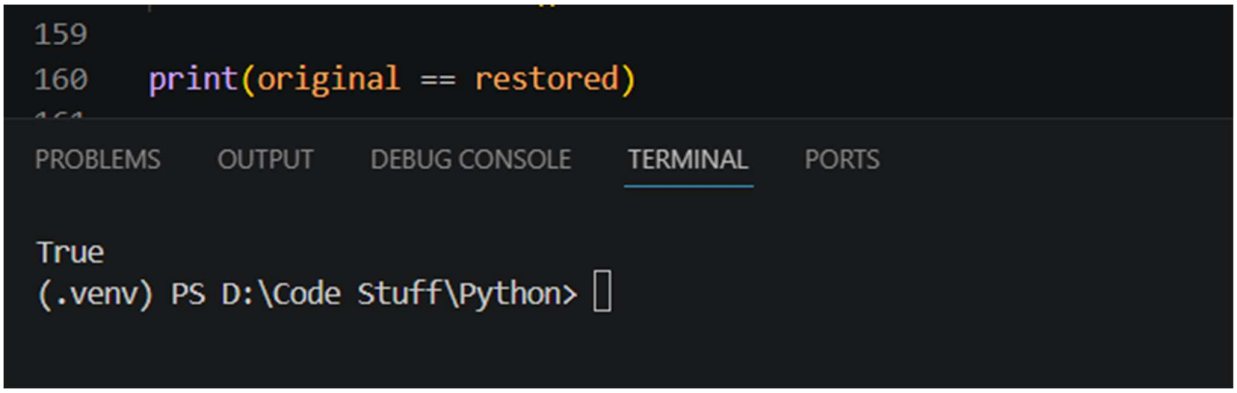
1. Encrypt the original text file.
2. Generate the ciphertext file.
3. Decrypt the ciphertext file.
4. Compare the decrypted file with the original file.

Verification Code:

original == restored

Result:

True



This confirms that the encryption and decryption processes are mathematically reversible and correctly implemented.

8.2 Test Cases

The implementation was tested using files of varying lengths and structures.

Test Case	Description	Result
Empty File	File containing no characters	Pass
Single Character	File containing one character	Pass
Two Characters	File containing two characters	Pass
Three Characters	File containing one complete block	Pass
Four Characters	File requiring padding	Pass
Paragraph Text	Multiple sentences	Pass

Test Case	Description	Result
Large Text File	Thousands of characters	Pass

All test cases successfully restored the original plaintext after decryption.

```python
if __name__ == "__main__":
    seed = 987654321

encrypt_file(
    "big.txt",
    "cipher.bin",
    seed
)

decrypt_file(
    "cipher.bin",
    "restored.txt"
)

os.system('cls')
with open("big.txt", "w") as f:
    f.write("Hello World\n" * 1_000_000)

with open("big.txt") as a:
    original = a.read()

with open("restored.txt") as b:
    restored = b.read()

print(original == restored)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

True
(.venv) PS D:\Code Stuff\Python>
```

8.3 Matrix Verification

Generated matrices were tested to ensure invertibility under modulo 257 arithmetic.

Validation Condition:

$\det(K) \neq 0 \pmod{257}$

Only matrices satisfying this condition were accepted by the system.

Result:

All generated matrices used during testing were successfully inverted and used for decryption.

8.4 Dynamic Matrix Generation Testing

The deterministic matrix generation process was tested using identical and different seed values.

Observations:

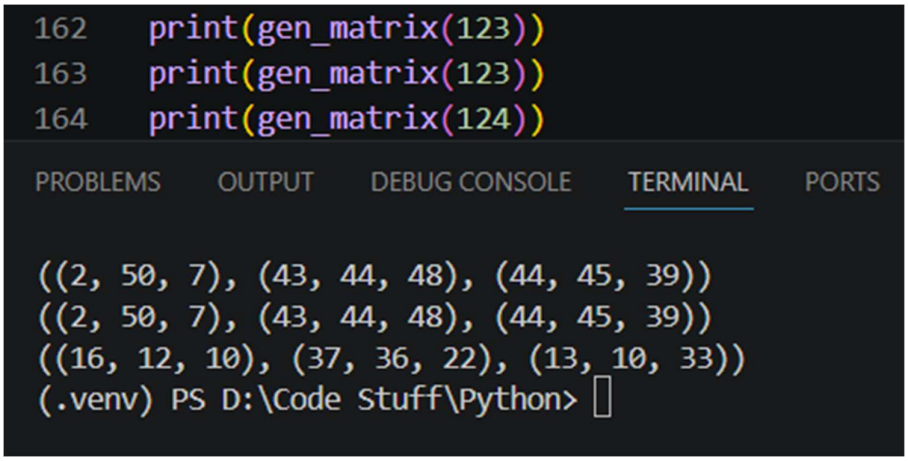
• Identical seeds always generated identical matrix sequences.

• Different seeds generated different matrix sequences.

• Block counters produced unique matrices for different blocks.

Result:

Deterministic reconstruction was successfully achieved during decryption.



8.5 Ciphertext Observation

The initial implementation using a single matrix exhibited visible ciphertext repetition when identical plaintext patterns appeared multiple times.

After introducing block-wise matrix generation:

• Ciphertext variability increased.

• Repetition became significantly less visible.

• Different plaintext blocks were encrypted using different matrices.

Result:

The final architecture produced more diverse ciphertext output compared to the initial implementation.

8.6 Memory Utilization

The implementation processes files incrementally rather than loading the entire file into memory.

Characteristics:

• Constant-size working blocks.

• Streaming file processing.

• Memory usage largely independent of file size.

Result:

The implementation successfully processed large files while maintaining low memory overhead.

8.7 Overall Results

The final implementation achieved all primary project objectives.

Summary:

- Successful encryption and decryption ✓
- Deterministic matrix generation✓
- Modular matrix inversion✓
- Block-wise matrix variation✓
- Reversible transformations✓
- Streaming file processing✓
- Successful recovery of original plain_text ✓

The final system demonstrates the practical implementation of matrix-based encryption concepts using modular arithmetic and deterministic key generation.

9. Limitations

While the project successfully demonstrates a reversible matrix-based encryption system, several limitations remain.

9.1 Educational Scope

The system was developed primarily as an educational and research-oriented project. The security of the algorithm has not been formally analysed or proven using modern cryptographic techniques.

9.2 Text File Support

The current implementation was tested primarily on text files. Additional work would be required to fully support arbitrary binary data and other file formats.

9.3 Absence of Public-Key Infrastructure

The current version assumes that communicating parties already possess the same seed value. Secure seed exchange mechanisms such as RSA have not yet been integrated into the implementation.

9.4 Limited Security Analysis

Although the final implementation reduces direct ciphertext repetition through block-wise matrix generation, the system has not been subjected to comprehensive cryptanalysis against known-plaintext, chosen-plaintext, or adaptive attacks.

9.5 Performance Evaluation

The project prioritizes correctness and reversibility over optimization. Formal benchmarking against established cryptographic systems such as AES was outside the scope of this work.

Despite these limitations, the project successfully demonstrates the practical implementation of matrix transformations, modular arithmetic, deterministic key generation, and reversible encryption concepts.

10. Future Work

Several potential improvements were identified during the development process.

10.1 RSA-Based Seed Exchange

A future version may incorporate RSA-based seed exchange to allow communicating parties to establish shared seeds securely without prior exchange.

Proposed Architecture:

RSA Public Key
↓
Encrypt Seed
↓
Transmit Seed
↓
Generate Matrix Sequence
↓
Encrypt File

10.2 Nonce-Based Session Randomization

A nonce (number used once) may be introduced for each encryption session.

Proposed Architecture:

Seed + Nonce
↓
Matrix Generation
↓
Encryption

This would ensure that identical files encrypted with the same seed produce different ciphertext outputs across different sessions.

10.3 Chaining-Based Frequency Mitigation

Future versions may implement ciphertext chaining mechanisms.

In such a design, each plaintext block would be influenced by previous ciphertext blocks, reducing observable repetition patterns.

10.4 Support for Binary Files

Future implementations may support images, audio files, compressed archives, and other binary formats.

10.5 Larger Matrix Dimensions

The current implementation uses 3×3 matrices. Future work may investigate larger matrix dimensions and higher-dimensional vector representations.

10.6 Advanced Mathematical Generators

Future research may explore alternative deterministic matrix generation methods, including techniques inspired by number theory, p-adic mathematics, and lattice-based structures.

10.7 Formal Security Analysis

A comprehensive cryptographic evaluation should be conducted to analyse resistance against known attacks and compare performance with established encryption standards.

11. Conclusion

This project explored the design and implementation of a custom educational encryption system inspired by vector-space transformations, modular arithmetic, and concepts related to lattice-based cryptographic research.

The system represents plaintext as three-dimensional vectors and applies reversible matrix transformations under modulo 257 arithmetic. Through multiple development iterations, the architecture evolved from a static matrix design to deterministic seed-based matrix generation and finally to block-wise matrix generation.

The implementation successfully achieved its primary objectives:

• Encryption and decryption of text files.

• Deterministic matrix generation from a shared seed.

• Reversible mathematical transformations.

• Block-wise matrix variation.

• Streaming file processing with low memory requirements.

Extensive testing demonstrated successful recovery of the original plaintext across multiple input sizes and edge cases, validating the correctness of the implementation.

Although the project does not claim cryptographic security comparable to modern standards such as AES or established post-quantum cryptographic systems, it provides a practical demonstration of how mathematical structures can be used to construct a functional encryption mechanism.

The project also highlights the challenges involved in designing secure encryption systems and provides a foundation for future exploration of hybrid encryption, nonce-based randomization, and advanced key-generation techniques.

Overall, the work successfully met its educational objectives while offering valuable insights into cryptographic design, implementation, and verification.